

처음부터 재현하는 방법

아무것도 없는 계정에서 이 평가를 그대로 돌리는 절차입니다. 아래 명령을 위에서부터 순서대로 복사해 붙이면 됩니다.
데이터베이스도, 실행 중인 API 서버도 필요 없습니다. LLM 호출만 가능하면 됩니다.

1. 준비물

	내용
git	저장소를 받습니다
uv	파이썬 3.13 과 의존성을 설치합니다 (https://docs.astral.sh/uv/)
LLM	OpenAI 호출 엔드포인트와 키

2. 저장소 받기와 코드 버전 고르기

```
`bash
git clone https://github.com/jexists/SKN30-FINAL-1Team-dev.git salesluv
cd salesluv
cp -R docs/eval/03_미팅브리핑 ../briefing-eval
`
```

평가 자료를 저장소 밖(../briefing-eval)으로 먼저 복사합니다. 아래에서 코드 버전을 바꾸면 docs/eval 폴더가 사라지기 때문입니다.

- 이 폴더의 결과(v13)를 그대로 재현할 때: `git checkout affc17d3`
- 지금 코드로 새로 짤 때: 이 단계를 건너웁니다. 프롬프트 버전이 다르면 점수도 달라집니다(8절).

```
`bash
git checkout affc17d3
`
```

3. 의존성 설치

```
`bash
cd backend
uv sync --frozen
export PYTHONPATH=.
`
```

- uv.lock 에 고정된 버전을 그대로 설치합니다. `pip install -e .` 은 패키지 구성이 없어 실패합니다.
- PYTHONPATH=. 가 없으면 스크립트가 app 모듈을 찾지 못합니다.
- 이후 명령은 모두 backend/ 에서, 이 값을 잡은 같은 셸에서 실행합니다.

4. 환경변수

```
`bash
```

```
cp .env.example .env
```

```
`
```

.env 를 열어 LLM 값만 채웁니다.

```
`
```

```
LLM_API_URL=
```

```
LLM_API_KEY=
```

```
LLM_MODEL=
```

```
`
```

- 필수 설정값은 APP_ENV 하나이고(예시 파일에 local 로 들어 있음), 나머지는 LLM 호출에 필요합니다.
- LLM_API_KEY 대신 OPENAI_API_KEY 를 써도 됩니다.
- DB 관련 값은 예시 그대로 뒤도 됩니다. 이 평가는 DB 를 건드리지 않습니다.

5. 스크립트와 데이터 넣기

```
`bash
```

```
cp ../../briefing-eval/스크립트/*.py scripts/
```

```
mkdir -p evaluation_data/briefing/v1
```

```
cp -R ../../briefing-eval/데이터셋/timelines evaluation_data/briefing/v1/
```

```
`
```

케이스(cases/)는 다음 단계에서 타임라인으로부터 다시 만듭니다. 폴더에 든 데이터셋/cases 와 생성 시각(manifest.json 의 generated_at)만 빼고 똑같이 나옵니다.

6. 실행

```
### (1) 타임라인 → 평가 케이스
```

```
`bash
```

```
for t in hanbit sejong daeyang miraero hanul cheongsan; do
```

```
  .venv/bin/python scripts/build_briefing_evaluation.py ₩
```

```
  --timeline evaluation_data/briefing/v1/timelines/$t ₩
```

```
  --out evaluation_data/briefing/v1/cases ₩
```

```
  --depths 1 3 5 10 30
```

```
done
```

```
`
```

체인 6개 × 깊이 5개 = **케이스 30개**. LLM 을 쓰지 않아 즉시 끝납니다.

(2) 골든셋 검사 — 브리핑 실행 전에 반드시 통과시킨다

```
`bash  
.  
venv/bin/python scripts/check_briefing_evaluation.py ₩  
--root evaluation_data/briefing/v1  
`
```

마지막 줄이 **전부 통과** 여야 합니다. LLM 을 쓰지 않습니다.

(3) 실제 브리핑 생성

```
`bash  
.  
venv/bin/python scripts/run_briefing_evaluation.py ₩  
--cases evaluation_data/briefing/v1/cases ₩  
--out evaluation_results/briefing/run-001  
`
```

30건에 약 15~20분, LLM 호출 30회. 한 건만 돌리려면 --only hanbit-d30 을 붙입니다.

(4) 채점

```
`bash  
.  
venv/bin/python scripts/judge_briefing.py ₩  
--cases evaluation_data/briefing/v1/cases ₩  
--run evaluation_results/briefing/run-001 ₩  
--out evaluation_results/briefing/run-001/judged  
`
```

LLM 호출 30회.

(5) 실행 결과 검사

```
`bash  
.  
venv/bin/python scripts/check_briefing_evaluation.py ₩  
--root evaluation_data/briefing/v1 ₩  
--run evaluation_results/briefing/run-001  
`
```

실행 기록·채점 결과까지 함께 검사합니다. 일부 케이스만 돌렸다면 "케이스 목록이 다름" 2건은 정상입니다.

(6) 집계와 비교

```
`bash  
.  
venv/bin/python scripts/summarize_briefing.py ₩  
--judged evaluation_results/briefing/run-001/judged ₩  
--out evaluation_results/briefing/run-001/SUMMARY.json  
.  
venv/bin/python -c "
```

```

import json

a = json.load(open('../..briefing-eval/결과/SUMMARY.json'))
b = json.load(open('evaluation_results/briefing/run-001/SUMMARY.json'))

for k in ['overall', *a['by_depth']]:
    x = a[k] if k == 'overall' else a['by_depth'][k]
    y = b[k] if k == 'overall' else b['by_depth'][k]
    d = y['percent_mean'] - x['percent_mean']

    print(f"\n{k:8} 기준 {x['percent_mean']:5.1f} 재현 {y['percent_mean']:5.1f} 차이 {d:+5.1f} RAG 미호출  

    {x['rag_min_failed']}→{y['rag_min_failed']}")

"
`

LLM 을 쓰지 않습니다.

```

7. DB 없이 도는 이유

브리핑 Agent 는 스냅샷에 `_report_scope` 가 있으면 DB 를 조회하고, 없으면 스냅샷 값을 그대로 돌려줍니다. 케이스의 `input.json` 에는 이 키를 넣지 않습니다.

보고서 원문 재조회처럼 DB 로 가는 경로가 하나 남아 있어서, 하네스가 그 조회 함수만 스냅샷에서 답하도록 바꿔 끼웁니다(`run_briefing_evaluation.py`).

그래서 도구는 실제로 호출되면서 값만 고정됩니다. 도구를 꺾다면 못 찾을 "필요할 때 과거 보고서를 찾아 읽는가" 까지 평가 대상으로 남습니다.

8. 결과가 달라질 수 있는 지점

- 같은 케이스라도 LLM 응답은 매번 조금씩 다릅니다. 절대 점수보다 **깊이별 경향** 을 봅니다.
 - 브리핑 프롬프트 버전이 바뀌면 점수가 달라집니다. 실행 기록의 `prompt_version` 으로 어떤 버전이었는지 확인합니다. 이 결과는 `contract_management.generate_briefing.v13` 입니다.
- 이후 저장소 코드가 바뀌었다면(예: develop 은 v15) 같은 케이스라도 점수가 달라질 수 있고, 그때는 **그 버전의 새 기준선** 으로 봅니다. v13 결과를 그대로 재현하려면 2절처럼 `git checkout affc17d3` 한 코드로 돌립니다.
- 첫 미팅(d01)은 LLM 을 타지 않고 고정 문구를 반환하므로 항상 같은 출력이 나옵니다.

9. 재현 확인 방법

아래 세 가지가 맞으면 같은 조건으로 돌아간 것입니다.

- (2)·(5) 검사가 **전부 통과**
- 실행 기록의 `prompt_version` 이 `contract_management.generate_briefing.v13`
- (6) 비교에서 **d01·d03·d05 평균 차이가 ±5%p 안**
d10·d30 평균은 크게 흔들립니다. 그 깊이에서는 모델이 과거 보고서 검색(RAG)을 부르지

말지가 실행마다 달라지고, 한 케이스에서 RAG 를 건너뛰면 그 케이스 점수가 40~50%p 떨어집니다.

케이스가 깊이당 6개라 평균이 20%p 이상 움직일 수 있습니다. 이 깊이는 평균 대신

RAG 미호출 건수와 그 케이스들의 점수 를 함께 봅니다.

이 문서대로 빈 폴더에서 처음부터 돌린 결과(2026-09-17, v13 코드 affc17d3)입니다.

깊이	기존	재현	차이	RAG 미호출
전체	89.8	86.5	-3.3	3 → 7
d01	95.1	97.0	+1.9	0 → 0
d03	98.0	98.0	+0.0	0 → 0
d05	95.8	93.1	-2.7	0 → 1
d10	79.9	57.4	-22.5	2 → 5
d30	80.4	86.9	+6.5	1 → 1

검사는 골든셋 2,128건, 실행 후 2,523건 모두 통과했습니다. 차이가 15%p 넘게 난 케이스 4건은 모두

두 실행 중 한쪽만 RAG 를 부른 경우였습니다(예: hanul-d10 기존 RAG 1회 94.1% → 재현 0회 41.2%).